

Plan de Pruebas

Sistema de Gestión Académica (SGA)

Proyecto: SGA – Backend

Framework: pytest (Python) + Bruno (API manual/automatizada)

Ubicación tests Python: `sga/backend/tests/`

Ubicación tests Bruno: `sga/backend/bruno/`

Resumen Ejecutivo

Categoría	Archivos / Colecciones	Casos de prueba
Dominio — tenant, usuarios, auth	<code>test_domain.py</code>	7
Dominio — matrícula	<code>test_matricula.py</code>	10
Dominio — periodos y oferta académica	<code>test_periodos_oferta.py</code>	12
Dominio + BD — ciclo académico	<code>test_academic_lifecycle.py</code>	23
Servicios (unitarias con mocks)	<code>test_services.py</code>	18
Integración HTTP (pytest + httpx)	<code>test_integration.py</code>	2
Bruno — API manual/automatizada	<code>bruno/</code>	22
Total	7 archivos / 1 colección	94

conftest.py es configuración de fixtures, no contiene casos de prueba.

1. Pruebas de Dominio — `test_domain.py`

Valida las reglas de negocio del dominio de tenants, usuarios y seguridad.

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
DOM-01	<code>test_validar_dominio_tenant_ok</code>	Unitaria	Nombre de dominio válido (uni-demo)	No lanza excepción
DOM-02	<code>test_validar_dominio_tenant_invalido</code>	Unitaria	Nombres inválidos (Uni_Demo — mayúsculas y guion bajo)	Lanza ValueError
DOM-03	<code>test_validar_escala_evaluacion_ok</code>	Unitaria	Escala válida: mínima=0, máxima=20, aprobatoria=11	No lanza excepción
DOM-04	<code>test_validar_escala_evaluacion_invalida</code>	Unitaria	Escala inválida: aprobatoria=25 supera máxima=20	Lanza ValueError
DOM-05	<code>test_admin_no_puede_crear_admin_central</code>	Unitaria	Rol ADMIN intenta crear usuario ADMIN_CENTRAL	Lanza ValueError de jerarquía
DOM-06	<code>test_cuenta_bloqueada_falso_si_no_hay_fecha</code>	Unitaria	<code>cuenta_bloqueada(None)</code> — sin fecha de bloqueo	Retorna False
DOM-07	<code>test_calcular_bloqueo_por_intentos</code>	Unitaria	<code>calcular_bloqueo(5)</code> — 5 intentos fallidos	Retorna objeto de bloqueo distinto de None

2. Pruebas de Matrícula — `test_matricula.py`

Valida las reglas de negocio del proceso de inscripción de alumnos.

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
MAT-01	test_periodo_en_matricula_ok	Unitaria	Periodo en estado MATRICULA	No lanza excepción
MAT-02	test_periodo_en_matricula_invalido	Unitaria	Periodo en estado CONFIGURACION	Lanza ValueError con mención de MATRICULA
MAT-03	test_seccion_sin_vacantes	Unitaria	Vacantes=0, estado ABIERTA	Lanza ValueError con mención de vacantes
MAT-04	test_seccion_cerrada	Unitaria	Vacantes=5, estado CERRADA	Lanza ValueError con mención de abierta
MAT-05	test_prerrequisito_no_cumplido	Unitaria	Curso requerido no está en los aprobados del alumno	Lanza ValueError: Prerrequisitos no cumplidos
MAT-06	test_prerrequisito_cumplido	Unitaria	Curso requerido sí está en los aprobados del alumno	No lanza excepción
MAT-07	test_exceso_creditos	Unitaria	Acumulado 16 + 4 nuevos > límite 18	Lanza ValueError: limite maximo
MAT-08	test_creditos_dentro_limite	Unitaria	Acumulado 4 + 4 nuevos ≤ límite 18	No lanza excepción
MAT-09	test_matricula_inactiva	Unitaria	Matrícula en estado RETIRADA	Lanza ValueError: activa
MAT-10	test_inscripcion_ya_retirada	Unitaria	Inscripción en estado RETIRADA	Lanza ValueError: activa

3. Pruebas de Periodos y Oferta Académica — test_periodos_oferta.py

Valida periodos académicos, prerrequisitos, pesos de evaluación y edición de secciones.

3.1 Validación de periodos

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
PER-01	test_validar_fechas_periodo_ok	Unitaria	Inicio 2026-03-01 < Fin 2026-07-15	No lanza excepción
PER-02	test_validar_fechas_periodo_invalido	Unitaria	Inicio 2026-07-15 > Fin 2026-03-01	Lanza ValueError: La fecha de inicio debe ser anterior a la fecha de fin.
PER-03	test_validar_transicion_estado_periodo_ok	Unitaria	CONFIGURACION→MATRICULA→REGISTRO_NOTAS→CERRADO	No lanza excepción en ninguna transición
PER-04	test_validar_transicion_estado_periodo_retroceder	Unitaria	MATRICULA→CONFIGURACION	Lanza ValueError: Transición

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
				ón no permitida
PER-05	test_validar_transicion_estado_periodo_salutar	Unitaria	CONFIGURACION→REGISTRO_NOTAS (salto de estado)	Lanza ValueError: Transición no permitida

3.2 Validación de prerequisites de curso

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
PRE-01	test_validar_prerrequisitos_curso_ok	Unitaria	Tipo APROBACION_CURSO con UUID de curso definido	No lanza excepción
PRE-02	test_validar_prerrequisitos_curso_invalido	Unitaria	Tipo APROBACION_CURSO sin UUID de curso	Lanza ValueError: se debe especificar el curso requerido
PRE-03	test_validar_prerrequisitos_creditos_ok	Unitaria	Tipo MINIMO_CREDITOS con valor=120	No lanza excepción
PRE-04	test_validar_prerrequisitos_creditos_invalido	Unitaria	Tipo MINIMO_CREDITOS con valor=0 o None	Lanza ValueError: se debe especificar un valor mayor a cero

3.3 Validación de pesos de componentes de evaluación

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
COM-01	test_validar_suma_pesos_componentes_ok	Unitaria	30% + 40% existentes + 30% nuevo = 100%	No lanza excepción
COM-02	test_validar_suma_pesos_componentes_excedido	Unitaria	30% + 40% existentes + 31% nuevo = 101%	Lanza ValueError: La suma de pesos supera el 100% permitido

3.4 Validación de edición de secciones

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
SEC-01	test_validar_edicion_seccion_ok	Unitaria	Periodo en estado CONFIGURACION	No lanza excepción
SEC-02	test_validar_edicion_seccion_invalida	Unitaria	Periodo en estado MATRICULA	Lanza ValueError: No se pueden crear ni modificar secciones

4. Pruebas del Ciclo Académico — test_academic_lifecycle.py

Valida la lógica de calificaciones, cálculos de notas y modelos de base de datos. Rama: feature/academic-lifecycle.

4.1 Validación de notas (TestValidateGradeValue)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
CAL-01	test_valid_boundary_values	Unitaria	Notas 0, 20 y 14.5 en escala 0-20	No lanza excepción
CAL-02	test_valid_0_to_100_scale	Unitaria	Notas 0, 100 y 72.5 en escala 0-100	No lanza excepción
CAL-03	test_above_max_raises	Unitaria	Nota 21 en escala 0-20 (supera máximo)	Lanza GradeDomainError
CAL-04	test_below_min_raises	Unitaria	Nota -0.01 en escala 0-20 (por debajo del mínimo)	Lanza GradeDomainError
CAL-05	test_exact_max_is_valid	Unitaria	Nota exactamente igual al máximo (20 en escala 0-20)	No lanza excepción

4.2 Cálculo de nota final (TestCalcNotaFinal)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
CAL-06	test_equal_weights	Unitaria	2 componentes al 50%: $(15 \times 50 + 13 \times 50) / 100$	Retorna Decimal("14.00")
CAL-07	test_three_component_weights	Unitaria	3 componentes: $12 \times 30\% + 16 \times 30\% + 14 \times 40\%$	Retorna Decimal("14.00")
CAL-08	test_weights_not_100_raises	Unitaria	Componentes suman 80% (no 100%)	Lanza CierreDomainError
CAL-09	test_empty_list_returns_zero	Unitaria	Lista vacía de calificaciones	Retorna Decimal("0.00")

4.3 Promedio ponderado (TestCalcPromediosPonderados)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
CAL-10	test_todos_includes_failed	Unitaria	Modo TODOS: incluye APROBADA (15,4cr) y DESAPROBADA (10,5cr)	Retorna Decimal("12.22")
CAL-11	test_solo_aprobados	Unitaria	Modo SOLO_APROBADOS: excluye DESAPROBADA	Retorna Decimal("15.00")
CAL-12	test_ultimo_keeps_latest_attempt	Unitaria	Modo ULTIMO: MAT01 reprobado reemplazado por aprobado	Retorna Decimal("15.56")
CAL-13	test_empty_list_returns_zero	Unitaria	Lista vacía con modo TODOS	Retorna Decimal("0.00")
CAL-14	test_only_active_entries_excluded	Unitaria	Inscripción en estado ACTIVA sin nota final	Retorna Decimal("0.00") — excluida del cálculo

4.4 Evaluación de políticas de condición académica (TestEvaluarPolitica)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
POL-01	test_mayor_que	Unitaria	MAYOR_QUE: $3 > 2 \rightarrow \text{True}$ / $2 > 2 \rightarrow \text{False}$	Booleano correcto en ambos casos

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
POL-02	test_mayor_igual	Unitaria	MAYOR_IGUAL: $2 \geq 2 \rightarrow \text{True}$ / $1 \geq 2 \rightarrow \text{False}$	Booleano correcto en ambos casos
POL-03	test_igual	Unitaria	IGUAL: $5=5 \rightarrow \text{True}$ / $4=5 \rightarrow \text{False}$	Booleano correcto en ambos casos
POL-04	test_menor_igual	Unitaria	MENOR_IGUAL: $2 \leq 3 \rightarrow \text{True}$ / $4 \leq 3 \rightarrow \text{False}$	Booleano correcto en ambos casos
POL-05	test_menor_que	Unitaria	MENOR_QUE: $1 < 2 \rightarrow \text{True}$ / $2 < 2 \rightarrow \text{False}$	Booleano correcto en ambos casos
POL-06	test_invalid_operator_raises	Unitaria	Operador desconocido "DISTINTO_DE"	Lanza CierreDomainError

4.5 Modelos de base de datos — SQLite en memoria (TestDatabaseModels)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
DB-01	test_create_tenant	Integración (BD)	Crear Tenant con dominio uni.edu.pe y hacer flush	Registro recuperable; fetched.dominio == "uni.edu.pe"
DB-02	test_create_escala_evaluacion	Integración (BD)	Crear EscalaEvaluacion vigesimal asociada a un tenant	nota_aprobatoria == 10.50 y nota_minima == 0.00
DB-03	test_base_metadata_includes_all_tables	Integración (BD)	Verificar 19 tablas registradas en Base.metadata	Ninguna tabla del conjunto esperado falta (missing vacío)

Tablas verificadas en DB-03: tenants, usuario, perfil_alumno, perfil_docente, periodo_academico, curso, seccion, inscripcion, matricula, componente_evaluacion, calificacion, correccion_nota, snapshot_promedio, condicion_academica_alumno, politica_condicion_academica, formula_promedio, cuenta_seguimiento_alumno, evento_academico, registro_auditoria.

5. Pruebas de Servicios — test_services.py

Unitarias con mocks de repositorios, JWT y hashing. Requieren @pytest.mark.asyncio.

5.1 Servicio de autenticación (AuthService)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
AUTH-01	test_auth_login_success	Unitaria (mock)	Login con credenciales válidas	access_token válido, rol == ADMIN, reset_login_attempts y audit_repo.registrar llamados
AUTH-02	test_auth_login_invalid_password	Unitaria (mock)	Login con contraseña incorrecta	Lanza UnauthorizedError: Credenciales invalidas.; _registrar_intento_fallido llamado
AUTH-03	test_auth_login_lockout	Unitaria (mock)	Login con cuenta bloqueada	Lanza ForbiddenError: Cuenta bloqueada temporalmente.
AUTH-04	test_auth_logout	Unitaria (mock)	Logout con JTI y EXP válidos	blacklist_token y audit_repo.registrar llamados una vez
AUTH-05	test_auth_get_me_success	Unitaria (mock)	Obtener usuario actual por ID	Retorna UserPublic con email, rol ALUMNO y activo=True
AUTH-06	test_auth_get_me_not_found	Unitaria (mock)	Repositorio retorna None para el ID dado	Lanza NotFoundError: Usuario no encontrado.

5.2 Servicio de periodos académicos (PeriodoService)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
SVC-PER-01	test_periodo_crear_success	Unitaria (mock)	Crear periodo 2026-I con fechas válidas	nombre_periodo == "2026-I", estado CONFIGURACION; repo.create y audit_repo.registrar llamados
SVC-PER-02	test_periodo_crear_invalid_dates	Unitaria (mock)	Fecha inicio posterior a fecha fin	Lanza ValidationError: La fecha de inicio debe ser anterior
SVC-PER-03	test_periodo_crear_conflict_name	Unitaria (mock)	Nombre 2026-I ya existe en el tenant	Lanza ConflictError: Ya existe un periodo con este nombre
SVC-PER-04	test_periodo_transicionar_success	Unitaria (mock)	Transición CONFIGURACION→MATRICULA sin conflictos	Estado retornado MATRICULA; repo.update_estado llamado
SVC-PER-05	test_periodo_transicionar_conflict_active	Unitaria (mock)	Ya existe otro periodo activo en el tenant	Lanza ConflictError: Ya existe un periodo activo

5.3 Servicio de oferta académica (OfertaService)

ID	Caso de prueba	Tipo	Descripción	Resultado esperado
SVC-OFE-01	test_oferta_crear_plan_estudios_success	Unitaria (mock)	Crear plan Ingeniería de Sistemas v2026 sin duplicados	carrera, version_plan y estado BORRADOR correctos
SVC-OFE-02	test_oferta_crear_plan_estudios_duplicate	Unitaria (mock)	Plan con misma carrera y versión ya existe	Lanza ConflictError: Ya existe una version de este plan
SVC-OFE-03	test_oferta_crear_curso_success	Unitaria (mock)	Crear curso SYS101 sin código duplicado	codigo_curso == "SYS101", nombre_curso correcto
SVC-OFE-04	test_oferta_crear_seccion_success	Unitaria (mock)	Crear sección A en periodo CONFIGURACION con 40 vacantes	codigo_seccion == "A", vacantes_maximas == 40
SVC-OFE-05	test_oferta_crear_seccion_invalid_period_state	Unitaria (mock)	Crear sección en periodo en estado MATRICULA	Lanza ValidationError: No se pueden crear ni modificar secciones
SVC-OFE-06	test_oferta_crear_componente_evaluacion_success	Unitaria (mock)	Componente 30% cuando ya acumulan 60% (total 90% ≤ 100%)	peso_relativo == Decimal("30.00")
SVC-OFE-07	test_oferta_crear_componente_evaluacion_exceeds_weight	Unitaria (mock)	Componente 30% cuando ya acumulan 80% (total 110% > 100%)	Lanza ValidationError: La suma de pesos supera el 100% permitido

6. Pruebas de Integración HTTP — test_integration.py

Pruebas de extremo a extremo con `httpx.AsyncClient` sobre la ASGI app real. Precondición: variable de entorno `RUN_INTEGRATION_TESTS=1` y Docker activo.

ID	Caso de prueba	Tipo	Endpoint	Resultado esperado
INT-01	test_health	Integración (HTTP)	GET /health	HTTP 200 · {"status": "ok"}
INT-02	test_login_admin_central	Integración (HTTP)	POST /api/v1/auth/login	HTTP 200 · respuesta contiene access_token y rol == "ADMIN_CENTRAL"

7. Pruebas de API con Bruno — sga/backend/bruno/

Rama: `feature/academic-lifecycle`. Herramienta: Bruno. Requiere servidor local en `{{baseUrl}}` con datos seed cargados. Las requests se ejecutan en orden secuencial ya que cada paso guarda variables de entorno para el siguiente.

7.1 Setup (00 Setup)

ID	Request	Método	Endpoint	Aserciones
BRU-01	Health	GET	/health	Status 200 · body.status == "ok"
BRU-02	Login ALUMNO (uni-demo)	POST	/api/v1/auth/login	Status 200 · body.rol == "ALUMNO" · guarda access_token

7.2 Flujo feliz — alumno (01 Flujo feliz (ALUMNO))

ID	Request	Método	Endpoint	Aserciones
BRU-03	GET /auth/me (guardar alumno_id)	GET	/api/v1/auth/me	Status 200 · guarda alumno_id
BRU-04	GET periodos (guardar id_periodo)	GET	/api/v1/periodos	Status 200 · guarda id_periodo
BRU-05	GET secciones del periodo	GET	/api/v1/periodos/{{id_periodo}}/secciones	Status 200 · guarda id_seccion
BRU-06	POST crear matrícula	POST	/api/v1/matriculas	Status 201 · body.estado == "ACTIVA" · guarda matricula_id
BRU-07	GET listar matrículas por alumno	GET	/api/v1/matriculas	Status 200 · body[0].id == {{matricula_id}}
BRU-08	POST inscribir curso	POST	/api/v1/matriculas/{{matricula_id}}/inscripciones	Status 201 · body.estado == "ACTIVA" · guarda inscripcion_id
BRU-09	GET inscripciones de la matrícula	GET	/api/v1/matriculas/{{matricula_id}}/inscripciones	Status 200 · body[0].id == {{inscripcion_id}}
BRU-10	POST retiro de inscripción	POST	/api/v1/inscripciones/{{inscripcion_id}}/retiro	Status 200 · body.estado == "RETIRADA"

7.3 Flujo admin — matrícula por alumno (02 Admin (matrícula por alumno))

ID	Request	Método	Endpoint	Aserciones
BRU-11	Login ADMIN (uni-demo)	POST	/api/v1/auth/login	Status 200 · body.rol == "ADMIN" · guarda access_token
BRU-12	GET matrículas del alumno (rol ADMIN)	GET	/api/v1/matriculas	Status 200

7.4 Errores esperados (03 Errores esperados)

ID	Request	Método	Endpoint	Aserciones
BRU-13	POST matrícula sin token → 401	POST	/api/v1/matriculas	Status 401 — sin header Authorization
BRU-14	POST matrícula duplicada → 409	POST	/api/v1/matriculas	Status 409 — alumno ya tiene matrícula en el periodo
BRU-15	POST retiro duplicado → 409	POST	/api/v1/inscripciones/{inscripcion_id}/retiro	Status 409 — inscripción ya fue retirada

7.5 Ciclo de vida académico (04 Ciclo de Vida Academico)

Flujo completo desde el registro de calificaciones hasta la descarga del récord académico en PDF. Rol DOCENTE para calificaciones, rol ADMINISTRADOR para cierre y consultas.

ID	Request	Método	Endpoint	Aserciones
BRU-16	01 Registrar Calificaciones	POST	/calificaciones/secciones/{id_seccion}/componentes/{id_componente}}	Sin aserciones automáticas; validación manual de respuesta
BRU-17	02 Publicar Componente	PUT	/calificaciones/secciones/{id_seccion}/componentes/{id_componente}/publicar	Sin aserciones automáticas
BRU-18	03 Corregir Calificacion	POST	/calificaciones/{id_calificacion}/corregir	Sin aserciones automáticas; justificación requerida
BRU-19	04 Cerrar Acta	POST	/cierre/secciones/{id_seccion}/cerrar-acta	Sin aserciones automáticas
BRU-20	05 Cerrar Periodo	POST	/cierre/periodos/{id_periodo}/cerrar	Sin aserciones automáticas
BRU-21	06 Obtener Historial	GET	/historial/alumnos/{id_perfil_alumno}}	Status 200 · body tiene propiedad periodos
BRU-22	07 Descargar PDF Record	GET	/historial/alumnos/{id_perfil_alumno}/pdf	Status 200 · Content-Type incluye application/pdf

8. Ejecución de las Pruebas

8.1 Pytest

```
# Todas las pruebas unitarias e integración BD (desde sga/backend/)
pytest tests/

# Solo pruebas de dominio
pytest tests/test_domain.py tests/test_matricula.py tests/test_periodos_oferta.py

# Solo ciclo académico
pytest tests/test_academic_lifecycle.py

# Solo servicios
pytest tests/test_services.py
```



```
# Integración HTTP (requiere Docker activo)
RUN_INTEGRATION_TESTS=1 pytest tests/test_integration.py

# Con reporte de cobertura
pytest tests/ --cov=app --cov-report=term-missing
```

8.2 Bruno

```
# Requiere Bruno CLI (bru)
bru run sga/backend/bruno/ --env "SGA Local"

# Solo una colección específica
bru run "sga/backend/bruno/04 Ciclo de Vida Academico/" --env "SGA Local"
```

9. Configuración CI/CD

Las pruebas pytest se ejecutan automáticamente en GitHub Actions mediante el archivo `.github/workflows/backend-tests.yml`. Las pruebas Bruno son manuales o pueden integrarse al pipeline con `bru run` en un step de CI.